

Übungsblatt: Vector Graphics

Empfohlene Bibliotheken

- NumPy
- svgwrite

Hinweise

- Öffne die erzeugten SVG-Dateien regelmäßig im Browser oder in Inkscape zur Kontrolle.
 - Achte auf eine saubere Struktur deiner SVG-Dateien.
 - Verwende nach Möglichkeit eigene Implementierungen der mathematischen Verfahren.
-

Aufgabe 1 – SVG-Datei manuell erstellen

Erstelle mit einem Texteditor eine SVG-Datei.

Anforderungen

Erzeuge ein einfaches Haus aus SVG-Primitiven:

Hauskörper

- Rechteck

Dach

- Dreieck (Polygon)

Tür

- Rechteck

Fenster

- Zwei Quadrate

Hinweise

Verwende ausschließlich SVG-Elemente wie:

- `<rect>`
 - `<polygon>`
-

Aufgabe 2 – SVG-Datei mit Inkscape bearbeiten

Installiere Inkscape und öffne die in Aufgabe 1 erzeugte SVG-Datei.

Anforderungen

1. Öffne die SVG-Datei in Inkscape.
 2. Verändere mindestens:
 - eine Farbe
 - die Position eines Elements
 - die Größe eines Elements
 3. Speichere die Datei erneut.
 4. Öffne die gespeicherte SVG-Datei anschließend wieder in einem Texteditor.
-

Aufgabe 3 – SVG-Dateien mit Python erzeugen

Erzeuge SVG-Dateien programmatisch mit der Bibliothek `svgwrite`.

Anforderungen

Erstelle eine SVG-Datei, die folgende Elemente enthält:

Linie

- Startpunkt frei wählbar
- Endpunkt frei wählbar

Kreis

- Radius frei wählbar

Polygon

- mindestens fünf Eckpunkte

Hinweise

Verwende die Bibliothek:

`svgwrite`

Aufgabe 4 – Quadratische Bézier-Kurve

Implementiere eine quadratische Bézier-Kurve.

Die Kurve wird beschrieben durch:

$$B(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} = (1-t)^2 \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + 2(1-t)t \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + t^2 \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$$

Anforderungen

1. Wähle drei Kontrollpunkte (P0, P1, P2).
2. Berechne die Kurve für:

$$t \in [0, 1]$$

mit mindestens 100 gleichmäßig verteilten Stützpunkten.

3. Speichere die Kurve als SVG-Datei.
-

Aufgabe 5 – Transformationen

Implementiere geometrische Transformationen.

Gegeben

Ein Dreieck mit frei wählbaren Eckpunkten.

Anforderungen

Implementiere folgende Transformationen:

Translation

Verschiebe das Dreieck.

Skalierung

Vergrößere oder verkleinere das Dreieck.

Rotation

Rotiere das Dreieck um einen frei wählbaren Winkel.

Darstellung

Zeige in einer SVG-Datei:

- das ursprüngliche Dreieck

- das transformierte Dreieck

in unterschiedlichen Farben.

Bonusaufgabe – Generative Vektorkunst

Erzeuge ein eigenes algorithmisches SVG-Kunstwerk.

Beispiele

- geometrische Muster
- rekursive Bäume
- Sternmuster
- Spiralen
- algorithmische Logos
- Fraktale

Bedingungen

Verwende mindestens zwei der folgenden Konzepte:

- Schleifen
- Zufall
- Rekursion
- Transformationen
- Bézier-Kurven

Ziel

Experimentiere mit den Möglichkeiten programmatisch erzeugter Vektorgrafiken.